COP 4020 Programming languages
University of Central Florida
Python Assignment: Employee management system



This assignment is about creating an employee management system where the program will read a data file and provide a command line user interface for various operations. The program will also print a report if user needs.

Employee Performance Tracking System

*1. **`Employee Class Methods`**:*

`__init__(self, idNum: int, name: str, dept: str, age: int, score: float, salary: int)`

- **Parameters**:
  - `idNum`: Integer representing the unique ID of the employee.
  - `name`: String representing the full name of the employee.
  - `dept`: String representing the department the employee belongs to.
  - `age`: Integer representing the age of the employee.
  - `score`: Float representing the employee's performance score.
  - `salary`: Integer representing the employee's salary.
- **Return Type**: `None` (constructor).

Initializes the variables based on the given parameters.

`createEmployee(record: str) -> Employee`

- **Parameters**:
  - `record`: A string representing a line of employee data (comma-separated) from the input file. Example: `"1001,Alice Johnson,Engineering,29,88,75000"`.
- **Return Type**: `Employee` object.

This method parses the `record` string, creates an instance of the `Employee` class, and returns it. This will be a static method.

`rating(self) -> str`

- **Parameters**:
  - `self`: The current `Employee` instance.
- **Return Type**: `str`.
  Returns the performance rating of the employee as a string:
  - "Excellent" if the score is above 90.
  - "Good" if the score is above 80.
  - "Fair" if the score is above 70.
  - "Bad" if the score is 70 or below.

`__lt__(self, other: Employee) -> bool`

- **Parameters**:
  - `self`: The current `Employee` instance.
  - `other`: Another `Employee` instance to compare with.
- **Return Type**: `bool`.
  Returns `True` if the `idNum` of the current employee is less than the `idNum` of the other employee, allowing for sorting by `idNum`.

`__eq__(self, other: Employee) -> bool`

- **Parameters**:
  - `self`: The current `Employee` instance.

- `other`: Another `Employee` instance to compare with.

- **Return Type**: `bool`.
  Returns `True` if all attributes of the current employee and the other employee are the same, indicating that they are equal.

`__repr__(self) -> str`

- **Parameters**:
  - `self`: The current `Employee` instance.
- **Return Type**: `str`.
  Returns a string representation of the employee's details, including their name, ID, department, rating, age, score, and salary. It returns a multi-line string in the following format:
  Name: <name>
  ID: <id>
  Department: <department>
  Rating: <rating>
  Age: <age>
  Score: <score>
  Salary: <salary>
  Use the same capitalization and labels as above. Score and Salary should be rounded exactly to 2 decimal.

*2. **Processor Class Methods**:*

`__init__(self, fname: str)`

- **Parameters**:
  - `fname`: A string representing the filename (e.g., `"data.txt"`) from which employee data will be read.
- **Return Type**: `None` (constructor).
  Initializes the processor, sets up an empty dictionary for employees, and calls `readFile()` to load the data. The file name and the employees dictionary will be instance variables of the Processor class.

`readFile(self) -> None`

- **Parameters**:
  - `self`: The current `Processor` instance.
- **Return Type**: `None`.
  Reads the employee data from the provided file, processes each line, creates `Employee` objects, and organizes them by department. This method will fill the employees dictionary. That dictionary will have department names as keys and list of employee objects in that department. If the file cannot be opened, the program must be printed exactly:
  cannot open file
  and then terminate (no further processing, no report).

`printAllEmployees(self) -> None`

- **Parameters**:
  - `self`: The current `Processor` instance.
- **Return Type**: `None`.
  Prints the details of all employees in the system. The employees are sorted by department and `idNum`. This method will be used in menu function. *__repr__()* function of employee class will be used while printing.

`findEmployee(self, searched: str) -> list[Employee]`

- **Parameters**:
  - `self`: The current `Processor` instance.
  - `searched`: A string representing the search term. The method will search for employees whose names contain this term (case-insensitive).
- **Return Type**: `list[Employee]`
  Returns a list of `Employee` objects whose names match the search term. If no employees found, this method will return an empty list.

`printSearchResult(self, searched: str) -> None`

- **Parameters**:
  - `self`: The current `Processor` instance.
  - `searched`: A string representing the search term to look for in employee names.
- **Return Type**: `None`.
  Prints the search result, displaying either the number of employees found or "No result" if no matches were found (meaning the findEmployees() function returned an empty list). *__repr__()* function of employee class will be used while printing.

`departmentStats(self, dept: str) -> str`

- **Parameters**:
  - `self`: The current `Processor` instance.
  - `dept`: A string representing the department for which the statistics will be generated.
- **Return Type**: `str`.
  Returns a formatted string with statistics for the specified department, including:
  - The number of employees in the department.
  - The average performance score of the department.
  - The average salary of the department.

`calculate_department_average(self, department: str, criteria: str) -> float`

- **Parameters**:
  - `self`: The current `Processor` instance.
  - `department`: A string representing the department for which the average is to be calculated.
  - `criteria`: A string that can be either `"score"` or `"salary"`, indicating which value to average (score or salary).
- **Return Type**: `float`.
  Returns the average of the specified `criteria` (score or salary) for the given department. The result is rounded to two decimal places.

`totalAvg(self, criteria: str) -> float`

- **Parameters**:
  - `self`: The current `Processor` instance.
  - `criteria`: A string that can be either `"score"` or `"salary"`, indicating which value to average across all departments.
- **Return Type**: `float`.
  Returns the average of the averages for the specified `criteria` (score or salary) across all departments. The result is rounded to two decimal places.

`generateReport(self) -> None`

- **Parameters**:
  - `self`: The current `Processor` instance.
- **Return Type**: `None`
  Generates a text report with company-wide and department-specific statistics and writes it to a file named "`report_<datafilename>`".

`menu(self) -> None`

- **Parameters**:
  - `self`: The current `Processor` instance.
- **Return Type**: `None`.

Starts an interactive menu where the user can:

1. Search for employees by name.
2. Generate a report.
3. Print all employees. The menu continues running until the user chooses to exit.
4. If user enters 4 the menu function finishes. Assume user will not enter any number besides 1,2,3,4.

All averages (department and company, performance and salary) must be printed with two digits after the decimal point.

Example of Method Interactions:

1. **Reading File**:
   - When the program starts, `Processor.__init__()` is called with the filename, which in turn calls `Processor.readFile()` to load employee data.
2. **Employee Search**:
   - The user can search for employees by name using the `Processor.printSearchResult()`, which internally calls `Processor.findEmployee()`.
3. **Generating Reports**:
   - The `Processor.generateReport()` method generates a comprehensive report containing company-wide statistics and department breakdowns.

Example data.txt

1001,Alice Johnson,Engineering,29,88,75000
1002,Bob Wilson,Marketing,34,76,68000
1003,Carol Davis,Engineering,41,95,85000
1004,David Brown,HR,28,72,52000
1005,Eva Martinez,Marketing,36,89,71000
1006,Frank Miller,Engineering,43,84,78000
1007,Kurt Wagner,Marketing,27,78.3,67000

Example Output run is given below. The user input is written bold.

Enter file name
data.txt
Choice 1 For searching users
Choice 2 For generating report
Choice 3 For printing all employees
Choice 4 For exiting
Enter choice
1
Enter searched string

avis
1 employee/s found
Name:Carol Davis
ID:1003
Department:Engineering
Rating:Excellent
Age:41
score:95.00
Salary:85000.00

Enter choice
2
Report generated

Enter choice
3
Name:Alice Johnson
ID:1001
Department:Engineering
Rating:Good
Age:29
score:88.00
Salary:75000.00

Name:Carol Davis
ID:1003
Department:Engineering
Rating:Excellent
Age:41
score:95.00
Salary:85000.00

Name:Frank Miller
ID:1006
Department:Engineering
Rating:Good
Age:43
score:84.00
Salary:78000.00

Name:David Brown
ID:1004
Department:HR
Rating:Fair
Age:28
score:72.00
Salary:52000.00

Name:Bob Wilson
ID:1002
Department:Marketing

Rating:Fair
Age:34
score:76.00
Salary:68000.00

Name:Eva Martinez
ID:1005
Department:Marketing
Rating:Good
Age:36
score:89.00
Salary:71000.00

Name:Kurt Wagner
ID:1007
Department:Marketing
Rating:Fair
Age:27
score:78.30
Salary:67000.00

All employees are printed

Enter choice
4
Done

Example output when the file cannot be opened
Enter file name
nonExistent.txt
cannot open file

report_data.txt is given below
==========
Generated from: data.txt

Company Statistics:
Total Employees: 7
Overall Average Performance: 80.70
Average Salary: 66666.67

Department Breakdown

Engineering Department:
- Employees: 3
- Average Performance: 89.00
- Average Salary: 79333.33

HR Department:
- Employees: 1
- Average Performance: 72.00

- Average Salary: 52000.00

Marketing Department:
- Employees: 3
- Average Performance: 81.10
- Average Salary: 68666.67

Hint: the main block will be as follows. The main block will be inside Processor.py file

```
# main method will create instance of Processor.
# then it will run menu method
if __name__ == "__main__":
    print("Enter file name")
    fname = input()
    employee_processor = Processor(fname)
    employee_processor.menu()
```

Rubric:

- For each test consider the examples as gold standard for formatting (spaces, new line, strings) and your output should match to that. On any mismatch points get deducted.
- Searching for employee test case:
    - Positive test (Employee exists in file): 10 pts
    - Negative test (Employee does not exist in file): 10 pts
- Printing all employees test case: 20 pts
    - If not sorted -10 pts will be deducted
- Printing report test case: 20 pts
- File not readable test case: 10 pts
- Comments and naming: 20 pts
    - There should be header comments for each file. Specify what the class/file is doing in a few sentences. Besides write your name surname and PID/NID
    - There should be also comments in the code. Provide header comments for the each function specifying what the function is doing in a few words.
- Files submitted correctly: 10 pts
    - You need to zip two python files. Do NOT put the python files inside a folder. Just zip the python files. The python files should have names Processor.py and Employee.py.